UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FCFM

Análisis Numérico

M.A. Maria Del Carmen Martínez Cejudo

Proyecto 3 Interpolación Lineal

Sebastián Calderón Carrillo 2087472

Edson Armando Amaya Macias 2151976

Juan José Trujillo Garza 1972407

Grupo 031

Introducción

Codificación

En esta sección se presentan y explican las funciones implementadas en el código, destacando cómo cada una traduce las propiedades matemáticas del método de bisección y cómo se integran para formar el flujo completo del programa.

evaluar_polimonio(coeficientes, x)

```
 99  def evaluar_polimonio(coeficientes: List[float], x: float) -> float:
100      """Evalúa un polinomio en un punto x dado sus coeficientes.
101      Los coeficientes se ordenan de mayor a menor grado.
102      Ejemplo: [1, -3, 2] representa x^2 - 3x + 2
103
104      Args:
105          coeficientes: Coeficientes del polinomio.
106          x: Valor de x a evaluar en el polinomio.
107
108      Returns:
109          float: Valor de la funcion evaluada en x.
110      """
111      resultado = 0.0
112      n = len(coeficientes)
113      for i, c in enumerate(coeficientes):
114          grado = n - 1 - i
115          resultado += c * (x ** grado)
116      return resultado
```

Explicación:
Esta función evalúa un polinomio en un punto x dado. Recibe una lista de coeficientes ordenados de mayor a menor grado y un valor x. Recorre los coeficientes y, para cada uno, calcula *c * (x ** grado)*, donde el grado se obtiene como *n - 1 - i (siendo n la cantidad de coeficientes).* Suma todos los términos y devuelve el resultado. Es la base para calcular *f(a), f(b) y f(c)* en cada iteración del método.

biseccion(coeficientes, lim_izq, lim_der, tolerancia=1e-6, max_iter=100)

```python
def biseccion(coeficientes: List[float], lim_izq: float, lim_der: float,
        tolerancia: float = 1e-6, max_iter: int = 100) -> Tuple[float, int, List[Tuple]]:
    """Implementa el método de bisección para encontrar una raíz.

    Args:
        coeficientes: Función continua
        lim_izq: Extremo izquierdo del intervalo
        lim_der: Extremo derecho del intervalo
        tolerancia: Tolerancia (criterio de parada)
        max_iter: Número máximo de iteraciones

    Returns:
        Tuple[float, int, List[Tuple]]: Tupla con (raíz, iteraciones, historial)

    Raises:
        ValueError: Si f(a) * f(b) >= 0 (no hay cambio de signo)
    """

    f_lim_izq = evaluar_polimonio(coeficientes, lim_izq)
    f_lim_der = evaluar_polimonio(coeficientes, lim_der)

    # Check if one of the fa and fb is actually a solution

    if f_lim_izq * f_lim_der >= 0:
        raise ValueError("No hay cambio de signo en el intervalo [a, b]. No se puede aplicar bisección.")

    historial = []

    for i in range(0, max_iter):
        punto_medio = (lim_izq + lim_der) / 2
        f_punto_medio = evaluar_polimonio(coeficientes, punto_medio)

        historial.append((i+1, lim_izq, lim_der, punto_medio, f_lim_izq, f_lim_der, f_punto_medio))

        if abs(f_punto_medio) <= tolerancia or (lim_der - lim_izq) / 2 <= tolerancia:
            return punto_medio, i, historial

        if f_lim_izq * f_punto_medio < 0:
            lim_der = punto_medio
            f_lim_der = f_punto_medio
        else:
            lim_izq = punto_medio
            f_lim_izq = f_punto_medio

    return punto_medio, max_iter, historial
```

Explicación:
Esta es la función principal del método. Primero evalúa el polinomio en los extremos lim_izq y lim_der. Si el producto *f(lim_izq) * f(lim_der) >= 0*, lanza una excepción ValueError indicando que no hay cambio de signo y que no se puede aplicar bisección. Si hay cambio de signo, inicializa una lista historial y comienza un ciclo de hasta max_iter iteraciones. En cada iteración:

1. *Calcula el punto medio punto_medio = (lim_izq + lim_der) / 2.*
2. *Evalúa f(punto_medio).*
3. *Agrega una tupla al historial con el número de iteración, los límites actuales, el punto medio, y los valores de f en los extremos y en el punto medio.*
4. *Verifica el criterio de parada: si abs(f(punto_medio)) <= tolerancia o si la mitad del ancho del intervalo (lim_der - lim_izq) / 2 <= tolerancia, devuelve el punto medio, el número de iteraciones realizadas (índice i) y el historial.*
5. *Si no se cumple el criterio, decide si la raíz está en [lim_izq, punto_medio] o en [punto_medio, lim_der] según el signo de f(lim_izq) * f(punto_medio), y actualiza los límites y los valores de f correspondientes.*

Si se alcanza el número máximo de iteraciones sin cumplir la tolerancia, devuelve el último punto medio calculado, max_iter y el historial completo. Esta función garantiza que el intervalo siempre contenga una raíz y que el error se reduzca a la mitad en cada paso.

ejecutar_biseccion()

```python
def ejecutar_biseccion():
    """Ejecuta el método de bisección con entrada del usuario."""
    print("\n--- Método de Bisección ---\n")

    # Leer coeficientes del polinomio
    print("Ingresa los coeficientes del polinomio (de mayor a menor grado)")
    print("Ejemplo: para x² - 3x + 2, ingresa: 1 -3 2")
    entrada = input("Coeficientes: ").strip().split()

    try:
        coeficientes = [float(c) for c in entrada]
    except ValueError:
        print("Error: Los coeficientes deben ser números.")
        return

    if len(coeficientes) == 0:
        print("Error: Debes ingresar al menos un coeficiente.")
        return

    # Leer intervalo [a, b]
    lim_izq = leer_real("Extremo izquierdo (a): ")
    lim_der = leer_real("Extremo derecho (b): ")

    if lim_izq >= lim_der:
        print("Error: 'a' debe ser menor que 'b'.")
        return

    # Leer tolerancia
    tolerancia = leer_real("Tolerancia (ej. 0.000001): ")

    try:
        raiz, iteraciones, historial = biseccion(coeficientes, lim_izq, lim_der, tolerancia)

        # Mostrar resultados
        print("\n=== Resultados ===")
        print(f"Raíz encontrada: {raiz}")
        print(f"Iteraciones: {iteraciones}")

        # Mostrar tabla de iteraciones
        print("\n--- Tabla de iteraciones ---")
        print(f"{'Iter':>4} {'a':>12} {'b':>12} {'c':>12} {'f(c)':>12}")
        print("-" * 56)
        for i, a_val, b_val, c, fa, fb, fc in historial:
            print(f"{i:>4} {a_val:>12.6f} {b_val:>12.6f} {c:>12.6f} {fc:>12.6f}")

    except ValueError as error:
        print(f"Error: {error}")
```

Explicación:
Esta función maneja la interacción con el usuario para el método de bisección. Muestra un mensaje explicativo, solicita los coeficientes del polinomio (separados por espacios, de mayor a menor grado), los límites izquierdo y derecho del intervalo, y la tolerancia. Valida que los coeficientes sean números reales y que al menos haya uno. También verifica que *lim_izq* < *lim_der*. Luego llama a la función biseccion y, si no hay errores, muestra la raíz encontrada, el número de iteraciones y una tabla formateada con el historial de iteraciones (columnas: Iter, a, b, c, f(c)). Si ocurre un ValueError (por ejemplo, por falta de cambio de signo), lo captura y muestra el mensaje de error. Esta función conecta la entrada del usuario con la lógica matemática y presenta los resultados de forma clara.

utils.py (funciones de utilidad)

Explicación:
Este módulo agrupa funciones auxiliares para la entrada/salida por consola:

- *leer_opcion(prompt, lower, upper): lee un entero dentro de un rango cerrado, repitiendo si hay error.*

```python
def leer_opcion(prompt:str, lower:int, upper:int) -> int:
    """Lee un entero dentro del rango cerrado [lower, upper]."""
    while True:
        entrada = input(prompt).strip()
        try:
            valor = int(entrada)
            if lower <= valor <= upper:
                return valor
            print(f"Opción fuera de rango (se esperaba entre {lower} y {upper}).")
        except ValueError:
            print(f'"{entrada}" no es un número entero válido.')
```

- *leer_entero(prompt): lee un entero cualquiera.*

```python
def leer_entero(prompt: str) -> int:
    """Lee un número entero del usuario."""
    while True:
        entrada = input(prompt).strip()
        try:
            return int(entrada)
        except ValueError:
            print(f'"{entrada}" no es un número entero válido.')
```

- *leer_real(prompt): lee un número real (float), repitiendo si hay error.*

```python
def leer_real(prompt: str) -> float:
    """Lee un número real del usuario."""
    while True:
        entrada = input(prompt).strip()
        try:
            return float(entrada)
        except ValueError:
            print(f'"{entrada}" no es un número real válido.')
```

- *borrar_consola(): limpia la pantalla según el sistema operativo (Windows: cls, otros: clear), pero solo si la entrada es una terminal interactiva (sys.stdin.isatty()).*

```python
def borrar_consola():
    """Detecta el sistema operativo y borra la consola."""
    if not sys.stdin.isatty():
        return
    os.system("cls" if os.name == "nt" else "clear")
```

- *pausar(): simula el comando pause de Windows o espera un Enter en otros sistemas, también solo en terminal interactiva.*

```python
def pausar():
    """Simula el comportamiento de pause en cmd."""
    if not sys.stdin.isatty():
        return
    if os.name == "nt":
        os.system("pause")
    else:
        input("Presiona Enter para continuar...")
```

Estas funciones garantizan que el programa no falle por entradas inválidas y que la experiencia de usuario sea fluida.

Integración de los módulos

El flujo completo del programa es el siguiente:

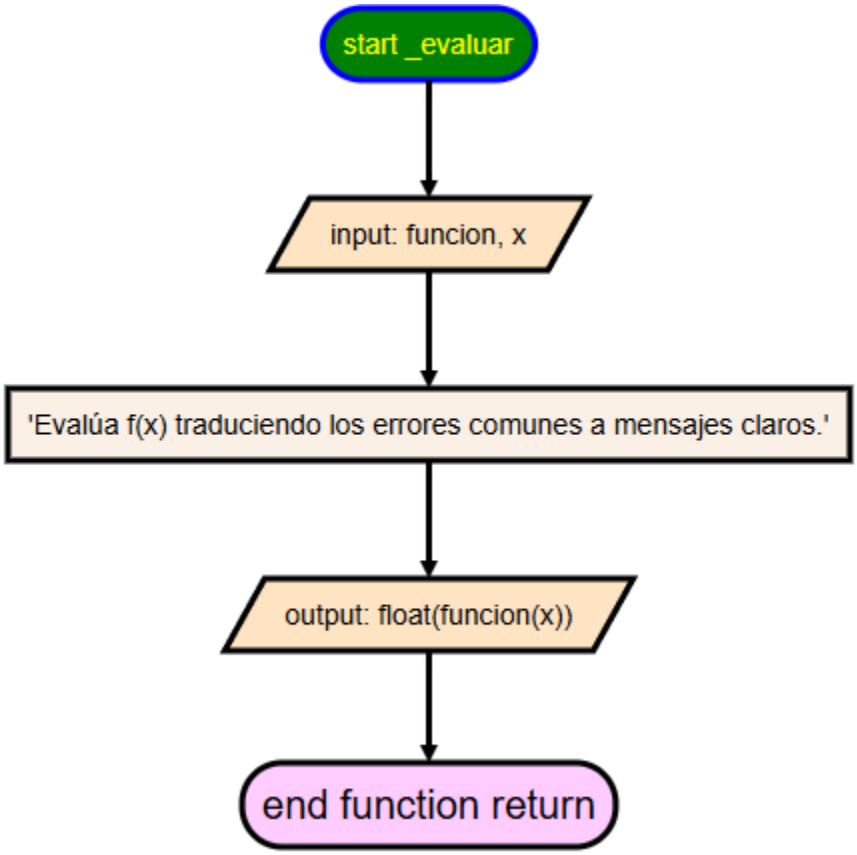
1. main.py (o el script de ejecución) llama a run() del módulo menu.
2. run() muestra el menú y, si se elige la opción 1, llama a ejecutar_biseccion() del módulo biseccion.
3. ejecutar_biseccion() solicita los datos al usuario usando leer_real de utils, valida los coeficientes y el intervalo, y llama a biseccion().

4. biseccion() utiliza evaluar_polimonio() para calcular los valores de f y ejecuta el algoritmo, devolviendo la raíz, el número de iteraciones y el historial.
5. ejecutar_biseccion() muestra los resultados y la tabla de iteraciones.
6. run() pausa la pantalla y vuelve al menú hasta que el usuario elija salir.

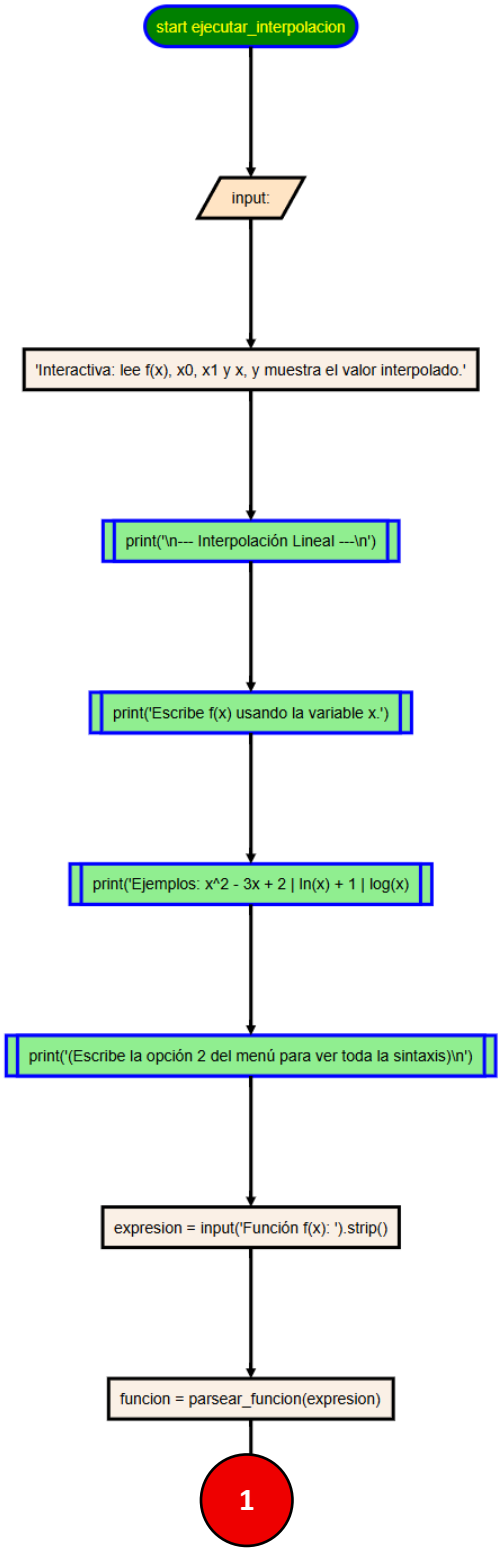
Esta separación en módulos facilita el mantenimiento, las pruebas unitarias y la reutilización del código en otros proyectos.

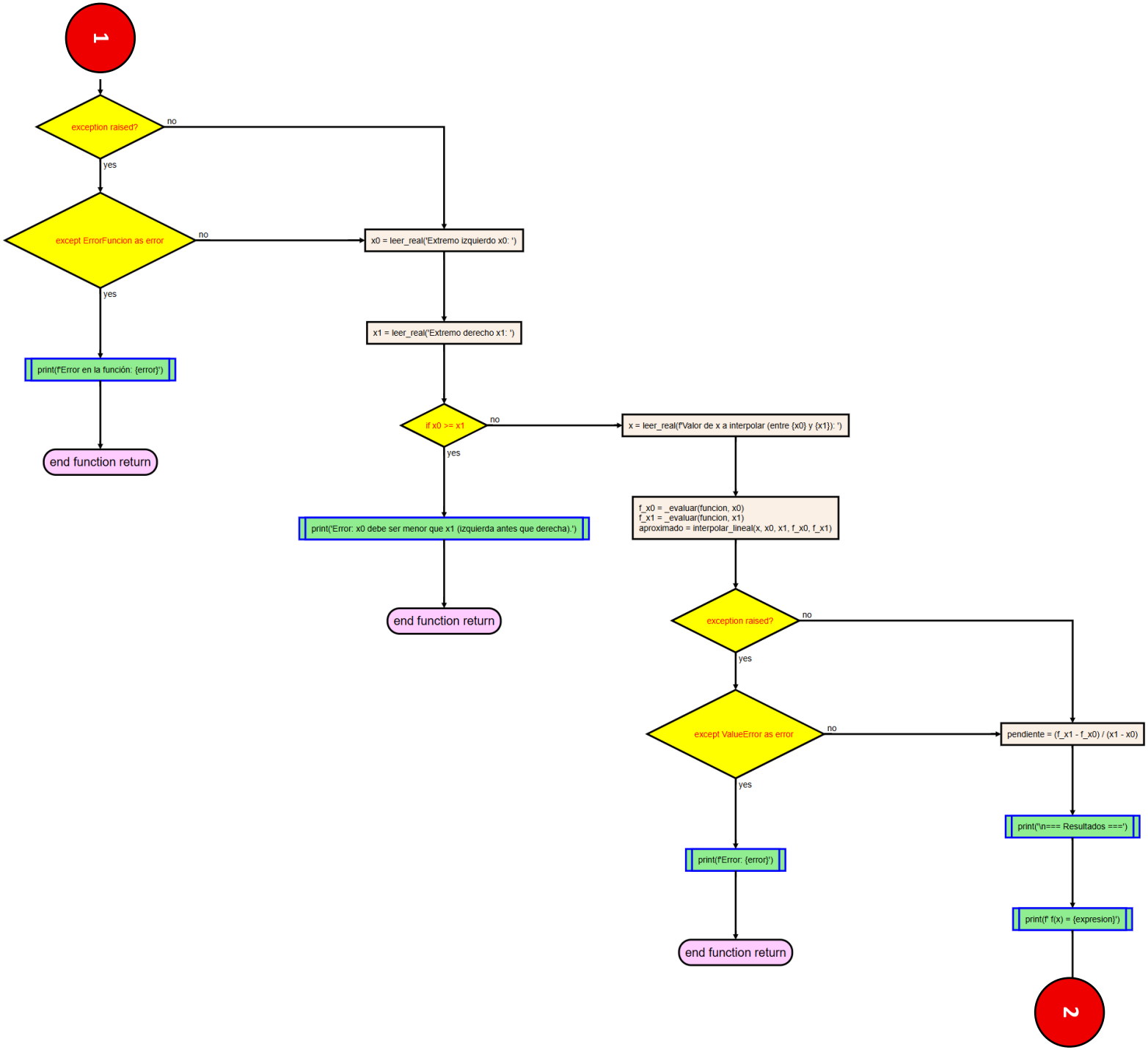
Diagramas de flujo

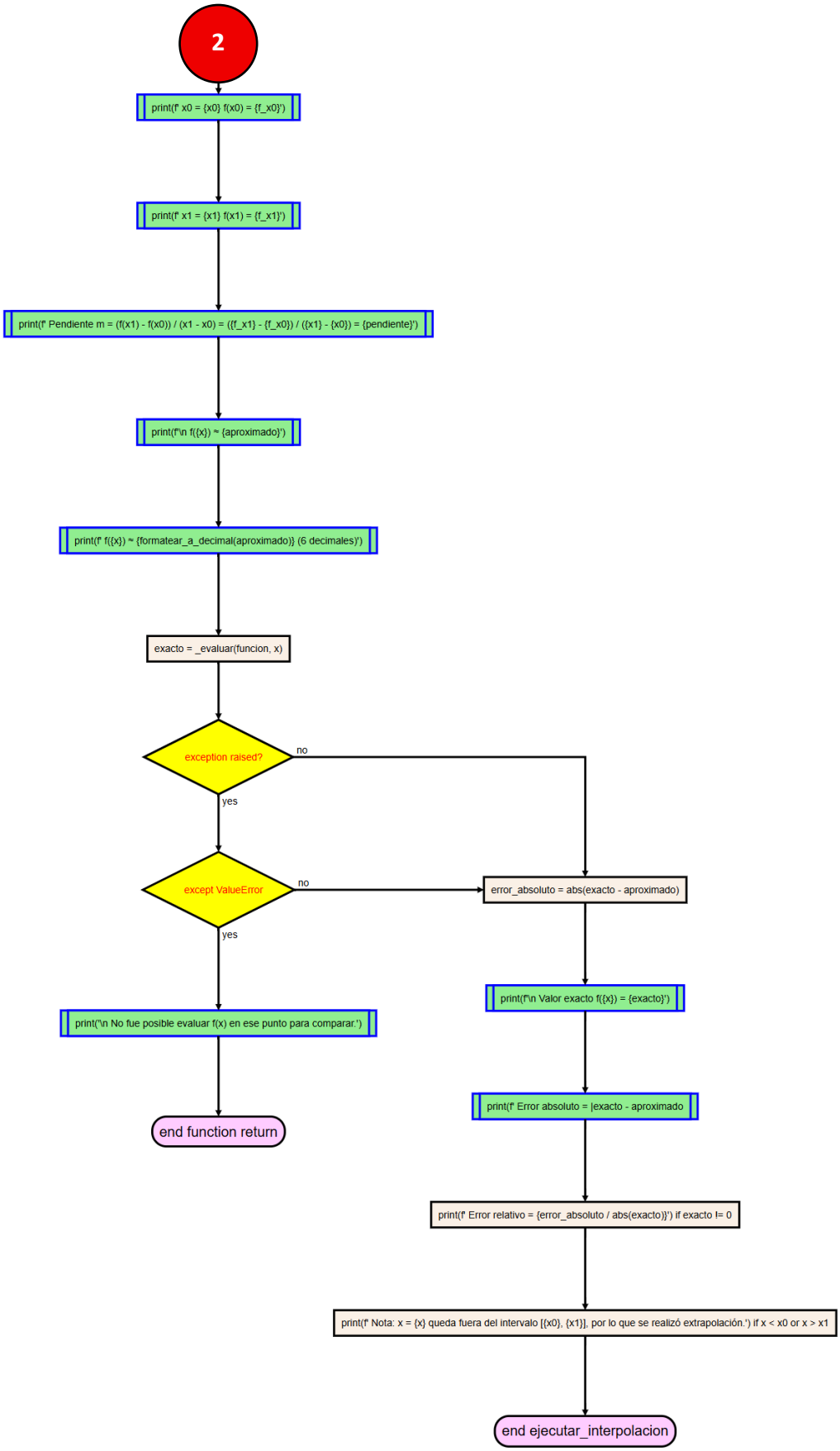
Evaluar Polinomio



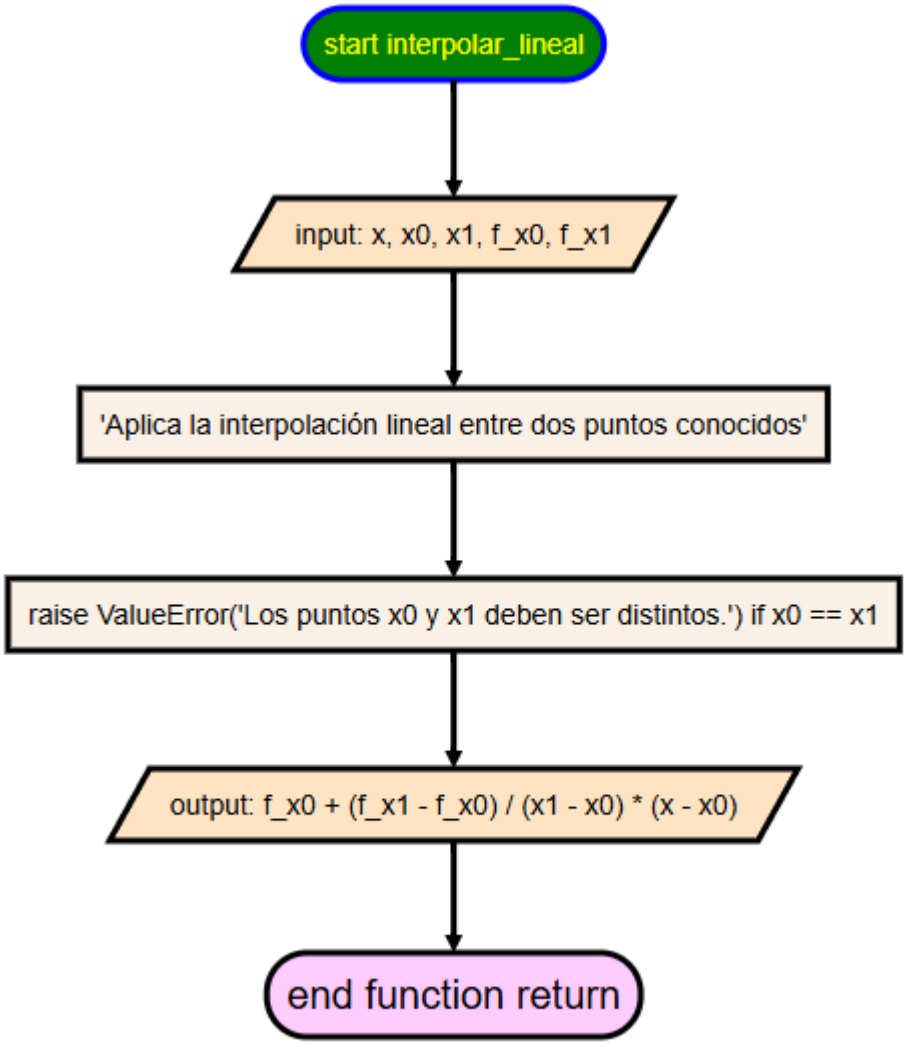
Ejecución de interpolación




1
exception raised?
no
yes
except ErrorFuncion as error
no
yes
print(f'Error en la función: {error}')
end function return
x0 = leer_real('Extremo izquierdo x0: ')
x1 = leer_real('Extremo derecho x1: ')
if x0 >= x1
no
yes
print('Error: x0 debe ser menor que x1 (izquierda antes que derecha).')
end function return
x = leer_real(f'Valor de x a interpolar (entre {x0} y {x1}): ')
f_x0 = _evaluar(funcion, x0)
f_x1 = _evaluar(funcion, x1)
aproximado = interpolar_lineal(x, x0, x1, f_x0, f_x1)
exception raised?
no
yes
except ValueError as error
no
yes
print(f'Error: {error}')
end function return
pendiente = (f_x1 - f_x0) / (x1 - x0)
print('\n=== Resultados ===')
print(f' f(x) = {expresion}')
2


2
print(f' x0 = {x0} f(x0) = {f_x0}')
print(f' x1 = {x1} f(x1) = {f_x1}')
print(f' Pendiente m = (f(x1) - f(x0)) / (x1 - x0) = ({f_x1} - {f_x0}) / ({x1} - {x0}) = {pendiente}')
print(f'\n f({x}) ≈ {aproximado}')
print(f' f({x}) ≈ {formatear_a_decimal(aproximado)} (6 decimales)')
exacto = _evaluar(funcion, x)
exception raised?
no
yes
except ValueError
no
yes
print('\n No fue posible evaluar f(x) en ese punto para comparar.')
end function return
error_absoluto = abs(exacto - aproximado)
print(f'\n Valor exacto f({x}) = {exacto}')
print(f' Error absoluto = |exacto - aproximado
print(f' Error relativo = {error_absoluto / abs(exacto)}') if exacto != 0
print(f' Nota: x = {x} queda fuera del intervalo [{x0}, {x1}], por lo que se realizó extrapolación.') if x < x0 or x > x1
end ejecutar_interpolacion

Interpolación de línea

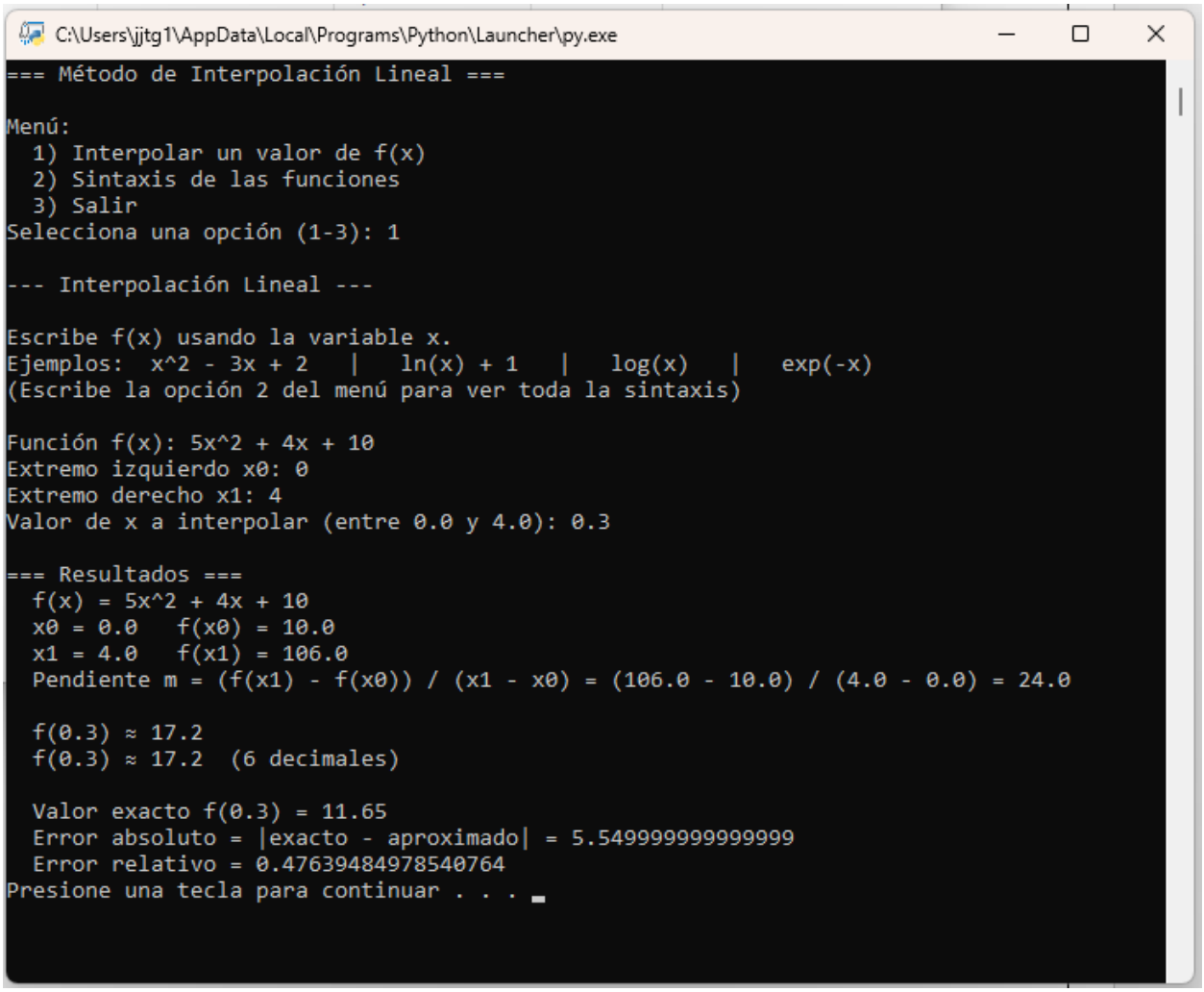


Conclusión general

El desarrollo de este proyecto permitió consolidar tanto la comprensión matemática como la aplicación práctica del Método de Bisección en la búsqueda de raíces de ecuaciones polinomiales. A través de la implementación algorítmica basada en el Teorema del Valor Intermedio (o Teorema de Bolzano), se comprobó la alta fiabilidad y la convergencia garantizada del método cuando se asegura un cambio de signo en los extremos del intervalo inicial. Si bien su convergencia es de carácter lineal y requiere un número mayor de iteraciones en comparación con métodos de mayor orden, su estabilidad lo convierte en una técnica fundamental e indispensable dentro del análisis numérico.

La automatización del algoritmo no solo elimina el margen de error humano y optimiza el cálculo iterativo de polinomios, sino que también reafirma el valor de integrar conceptos matemáticos rigurosos con buenas prácticas de programación para la resolución eficiente de problemas científicos e ingenieriles

Ejemplo de ejecución



Presentación



Problematica

Los centros de datos cuentan con un banco de **baterías de respaldo (UPS)** que almacenan energía o la entregan según la demanda.

En la realidad, se tienen medidas discretas de potencias tomadas por sensores a ciertas horas del dia

Potencia positiva(+): Las baterias se cargaron con el excedente de energia
Potencia negativa(-): Se descargaron las baterias para mantener al servidor.

Nosotros calculamos en que medida se presenta una **Potencia positiva o negativa** en base a 3 o mas metricas discretas del comportamiento a las diversas horas.

Eleccion de los metodos

Integracion con Trapecios

La aproximación geométrica por trapecios procesa de forma nativa los cambios de signo de la función f(t). Las áreas por encima del **eje x (carga)** suman y las **áreas por debajo (descarga**) restan de forma automática, entregando el cambio neto en kWh sin perder exactitud en los subintervalos Δt

Metodo Montante

Para ajustar los datos discretos de los sensores a una función polinómica continua f(t) mediante una **matriz de Vandermonde.**

Conclusion

Los sensores registran mediciones de potencia aisladas (positiva al cargar, negativa al descargar las baterías UPS). Para resolver esta falta de continuidad operativa, el proyecto aplica el método de Montante para ajustar un polinomio exacto y la regla del trapecio para integrarlo, obteniendo el balance neto real en kWh.